Intelligent Research Topic Analysis & Agentic AI Research Assistant

Milestone 1: Traditional NLP Research Analysis System

Mid-Semester Project Report

Team Members:

Kushal Sarkar
Chinmay Soni
Lakshya Bapna

Course: Generative AI (Section A)

Submission: Mid-Semester Evaluation

Date: March 2026

Application: github.com/Kushal425/GENai_SecA_P1

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Problem Statement 2
2.2 Objectives 2
2.3 Scope of Milestone 1 2

3 System Architecture **3**
3.1 Input Specification 3
3.2 Output Specification 4

4 Technical Implementation **4**
4.1 Text Preprocessing 4
4.2 Feature Extraction (TF-IDF) 4
4.3 Keyword Extraction 5
4.4 Topic Modeling (LDA) 5
4.4.1 Topic Coherence Evaluation 5
4.5 Extractive Summarization 5
4.6 Document Loading 6
4.7 User Interface & Visualizations 6

5 Results & Evaluation **6**
5.1 Sample Output 6
5.2 Evaluation Metrics 7

6 Limitations **7**

7 GitHub Repository & Code Quality **7**
7.1 Project Structure 8

8 Conclusion **8**

References **9**

1 Abstract

This report presents the design and implementation of **ResearchScope**, an intelligent research document analysis system developed as part of the Generative AI course (Milestone 1). The system employs a classical, statistics-based Natural Language Processing (NLP) pipeline to process research documents and produce structured analytical summaries — strictly without the use of any Large Language Models (LLMs) or agentic AI frameworks. The pipeline accepts raw text or uploaded documents (PDF/TXT), performs text preprocessing, applies TF-IDF based keyword extraction, discovers latent topics using Latent Dirichlet Allocation (LDA), and generates extractive summaries using sentence-level TF-IDF scoring. Results are presented through an interactive Streamlit web interface with multiple visualizations. This report documents the problem formulation, technical architecture, implementation details, evaluation results, and known limitations of the current approach.

2 Introduction

2.1 Problem Statement

The exponential growth of academic literature makes it increasingly difficult for researchers to efficiently identify key themes, extract important terminology, and understand document collections without spending hours reading. A research analysis system that can automatically surface key vocabulary, cluster related concepts, and generate concise summaries would drastically reduce this cognitive overhead.

2.2 Objectives

The objectives of Milestone 1 are to:

1. Build a functional NLP pipeline that accepts research text or document uploads.
2. Identify key vocabulary using statistical feature extraction (TF-IDF).
3. Discover latent topics using probabilistic topic modeling (LDA).
4. Generate extractive summaries using sentence-level scoring.
5. Present all outputs through a clean, interactive web interface.
6. Strictly adhere to classical ML/NLP methods – no LLMs are used.

2.3 Scope of Milestone 1

Milestone 1 focuses exclusively on a **Traditional NLP pipeline**. The system operates purely on statistical patterns and classical machine learning. Semantic understanding, reasoning, and agentic workflows are reserved for Milestone 2.

3 System Architecture

The system follows a sequential data processing pipeline, illustrated below. Each stage is encapsulated in a dedicated Python module for maintainability and testability.

```
Input (Text / PDF / TXT)
      |
      v
Text Preprocessing          [src/preprocessing.py]
  - Tokenization
  - Stopword Removal
  - Lemmatization (spaCy)
      |
      v
Feature Extraction          [src/feature_extraction.py]
  - TF-IDF Vectorization (scikit-learn)
      |
      +-------------------+
      |                   |
      v                   v
Keyword Extraction    Topic Modeling         [src/topic_model.py]
[src/keyword_         - LDA (gensim)
 extractor.py]        - Coherence Eval       [src/evaluation.py]
      |
      v
Extractive Summarization    [src/summarizer.py]
  - Sentence Scoring & Ranking (TF-IDF)
      |
      v
UI & Visualizations         [app.py, src/visualizations.py]
  - Streamlit Dashboard
  - Word Cloud, Bar Charts
```

Figure 1: ResearchScope System Architecture – Milestone 1 NLP Pipeline

3.1 Input Specification

- **Raw Text:** User can paste any research text directly into the Streamlit text area.
- **Document Upload:** One or more `.pdf` or `.txt` files can be uploaded; they are parsed and concatenated into a single corpus.

3.2 Output Specification

Table 1: System Outputs – Milestone 1

Output	Description
Keywords	Top-N terms ranked by TF-IDF score
Topic Clusters	N LDA-discovered latent themes with top words and probability weights
Coherence Score	C_v metric quantifying topic model quality
Extractive Summary	Top-N high-scoring sentences in original document order
Visualizations	Word cloud, keyword bar chart, LDA topic word distribution chart

4 Technical Implementation

4.1 Text Preprocessing

Module: `src/preprocessing.py` **Libraries:** `nltk`, `spacy`

The preprocessing stage transforms raw input into a clean, normalized token sequence through the following steps:

1. **Lowercasing:** All characters are converted to lowercase to unify case variants.
2. **Punctuation Removal:** Non-alphanumeric characters are removed using regular expressions.
3. **Tokenization:** The text is segmented into individual word tokens using `nltk.word_tokenize()`.
4. **Stopword Removal:** Common, semantically uninformative words (e.g., "the", "and", "is") are filtered using NLTK's English stopword corpus. Non-alphabetic tokens are also discarded.
5. **Lemmatization:** Tokens are reduced to their canonical dictionary form using spaCy's `en_core_web_sm` model (e.g., "running" → "run", "algorithms" → "algorithm"). This reduces vocabulary dimensionality and improves model quality.

A secondary utility function `get_text_stats()` computes surface-level statistics (sentence count, word count, unique token count) from the raw text for display in the application's metrics row.

4.2 Feature Extraction (TF-IDF)

Module: `src/feature_extraction.py` **Library:** `scikit-learn`

Term Frequency-Inverse Document Frequency (TF-IDF) is used to convert the preprocessed text corpus into a numerical feature matrix. For a term t in document d within corpus D:

$$\text{TF-IDF}(t, d, D) = \underbrace{\frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}}_{\text{Term Frequency}} \times \underbrace{\log\left(\frac{|D|}{|\{d \in D : t \in d\}|}\right)}_{\text{Inverse Document Frequency}} \tag{1}$$

Terms that are frequent within the document but rare across the corpus receive high weights, making them effective discriminative keywords. The implementation uses `TfidfVectorizer(ma` `min_df=1)` from scikit-learn to support single-document analysis.

4.3 Keyword Extraction

Module: `src/keyword_extractor.py` **Library:** `numpy`

Keywords are extracted by summing TF-IDF scores across all rows of the feature matrix for each vocabulary term, and selecting the top-N terms by aggregated score. This produces a ranked list of the most discriminative vocabulary items in the corpus.

4.4 Topic Modeling (LDA)

Module: `src/topic_model.py` **Library:** `gensim`

Latent Dirichlet Allocation (LDA) is a generative probabilistic model that discovers abstract "topics" from a document corpus. It assumes that each document is a mixture of topics, and each topic is a distribution over words.

The implementation:

1. Tokenizes the preprocessed corpus into word lists.
2. Constructs a vocabulary dictionary (`corpora.Dictionary`).
3. Converts documents to Bag-of-Words (BoW) vectors.
4. Fits an `LdaModel` with user-configurable number of topics (2–10, default: 5) over 10 training passes.

4.4.1 Topic Coherence Evaluation

Module: `src/evaluation.py`

Model quality is evaluated using the C_v coherence score (Röder et al., 2015), implemented via `gensim.models.CoherenceModel`. This metric measures the semantic similarity of high-scoring words within each topic using a sliding window over aggregated co-occurrence counts. Scores range from 0 to 1, with higher values indicating more interpretable, human-understandable topics.

4.5 Extractive Summarization

Module: `src/summarizer.py` **Libraries:** `nltk`, `scikit-learn`

The summarization approach is purely **extractive** – no text is generated; instead, the most informative existing sentences are selected:

1. The raw text is segmented into individual sentences using `nltk.sent_tokenize()`.

2. Each sentence is vectorized independently using `TfidfVectorizer`.
3. Sentences are scored by summing their TF-IDF term weights.
4. The top-N highest-scoring sentences are selected (user-configurable, default: 5).
5. Selected sentences are re-ordered to their **original document position** before output, preserving narrative coherence.
6. Edge cases (single-sentence input, very short texts) are handled with appropriate fallbacks.

4.6 Document Loading

Module: `src/document_loader.py` **Library:** `PyPDF2`

The system supports multi-format document intake:

- **PDF files:** Text is extracted page-by-page using `PyPDF2.PdfReader`.
- **TXT files:** Content is decoded using UTF-8 encoding with error tolerance.
- Multiple uploaded files are concatenated into a unified corpus for joint analysis.

4.7 User Interface & Visualizations

Module: `app.py`, `src/visualizations.py` **Libraries:** `streamlit`, `matplotlib`, `wordcloud`, `pandas`

The Streamlit interface provides:

- **Sidebar Controls:** Input mode toggle (paste text or upload), LDA topic count slider (2–10), keyword count slider (5–30), summary sentence count slider (2–10).
- **Metrics Row:** Live statistics (document count, word count, sentence count, unique vocabulary).
- **Four Result Tabs:** Keywords (TF-IDF table + badge display), Topics (LDA expanders with coherence score), Summary (extractive output with sentence scores), Visualizations (word cloud, keyword bar chart, topic distribution chart).

All plots use a dark-mode color scheme (`plasma` colormap) consistent with the Streamlit dark theme.

5 Results & Evaluation

5.1 Sample Output

The system was tested with a sample research text on Natural Language Processing. The pipeline successfully:

- Extracted keywords including “document”, “language”, “computer”, “collection”, and “frequency” with non-zero TF-IDF scores.

- Identified 5 distinct LDA topics with a coherence score of $C_v = 1.0$ (indicative of small, focused single-document input).
- Generated a 3-sentence extractive summary preserving the original document order.

5.2 Evaluation Metrics

Table 2: Evaluation Results on Sample Input

Metric	Method	Result
Keyword Relevance	TF-IDF Score Ranking	Top 15 terms extracted
Topic Quality	LDA Coherence (C_v)	1.0 (single-document)
Summary Coverage	Sentence TF-IDF Scoring	3–5 sentences selected
Pipeline Stability	End-to-end unit test	Passed (no errors)

6 Limitations

The following limitations of the traditional NLP approach are inherent to the classical methods used and directly motivate the transition to an Agentic AI system in Milestone 2:

1. **Lack of Semantic Understanding:** TF-IDF and LDA rely purely on word frequency and co-occurrence statistics. They cannot understand meaning, context, or synonyms. For example, "automobile" and "car" are treated as completely distinct terms despite identical meaning.
2. **Rigid Extractive Summaries:** The summarizer selects verbatim sentences from the source document. If a selected sentence relies on surrounding context (e.g., "He conducted the experiment..."), the output may be difficult to follow without the surrounding text.
3. **Fixed Topic Constraints:** LDA requires the user to pre-specify the number of topics. An incorrect choice either merges unrelated concepts into single topics or unnecessarily splits coherent themes, degrading interpretability.
4. **Static, Single-Pass Processing:** The system processes only the input provided at runtime. It cannot reason across multiple independent research papers, retrieve additional context, or iteratively refine its understanding over time.
5. **Language & Domain Specificity:** The pipeline relies on English-specific stopword lists and spaCy's English language model. Performance will degrade significantly on non-English text or highly domain-specific scientific jargon not represented in the model's vocabulary.

7 GitHub Repository & Code Quality

- **Repository:** https://github.com/Kushal425/GENai_SecA_P1
- **Branch:** `Kushal`

- **Structure:** The project follows a modular design with all NLP components separated into individual files under `src/`, minimizing coupling and maximizing reusability.
- **Code Quality:** Functions are named descriptively, each module has a single responsibility, and all edge cases (short texts, single sentences, empty uploads) are handled gracefully.
- **Documentation:** A comprehensive `README.md` with 6-step installation and local hosting instructions, technology stack, milestone roadmap, architecture overview, and limitations section.
- **Dependencies:** All required packages are listed in `requirements.txt`. A `setup.sh` script automates the entire environment setup process.

7.1 Project Structure

```
ResearchScope/
|
|-- data/                          # Input documents and datasets
|-- src/
|   |-- preprocessing.py           # Text cleaning, tokenization,
   lemmatization
|   |-- feature_extraction.py      # TF-IDF vectorization
|   |-- topic_model.py             # LDA topic modeling
|   |-- evaluation.py              # Coherence score evaluation
|   |-- keyword_extractor.py       # Keyword extraction logic
|   |-- summarizer.py              # Extractive summarization
|   |-- document_loader.py         # PDF and TXT file loading
|   `-- visualizations.py          # Word cloud, bar charts
|
|-- app.py                         # Main Streamlit application
|-- requirements.txt               # Python dependencies
|-- setup.sh                       # One-command environment setup
`-- README.md                      # Project documentation
```

8 Conclusion

ResearchScope Milestone 1 successfully demonstrates a functional, modular, and well-structured traditional NLP research analysis system. All core requirements of the mid-semester evaluation have been implemented: text preprocessing (tokenization, stopword removal, lemmatization), TF-IDF feature extraction, LDA topic modeling with coherence evaluation, extractive summarization, a multi-format document loader, and an interactive Streamlit UI with visualizations.

The limitations identified in this report — particularly the lack of semantic understanding and the inability to reason across document collections — form the precise motivation for extending the system into a full Agentic AI Research Assistant in Milestone 2, where LLM integration, Retrieval-Augmented Generation (RAG), and LangGraph-based workflow orchestration will directly address these shortcomings.

References

1. Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3, 993–1022.
2. Röder, M., Both, A., & Hinneburg, A. (2015). Exploring the space of topic coherence measures. *Proceedings of WSDM '15*, 399–408.
3. Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21. (TF-IDF foundation)
4. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
5. Honnibal, M., & Montani, I. (2017). spaCy 2: Natural Language Understanding with Bloom Embeddings, Convolutional Neural Networks and Incremental Parsing. *To appear*.
6. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.